COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	8/27/26	**DATE SUBMITTED:**	8/27/26
NAME:	Payas, John Paul S	**INSTRUCTOR:**	Sir. Jimlord Quejado

ACTIVITY NO. 6 **Searching Techniques**

1. PROCEDURE OUTPUT

OUTPUT

```
8571 10685 10649 16651 21206 32134 25583 10550 11277 14068 31146 28209 26066 7617 32304 22048 27848 28373 29269 30674 28821 32523 27669 5024 26937 31544 26249
29591 1694 26986 19988 5849 15983 20333 7390 11328 27694 8852 7043 8261 14209 25781 6668 16928 13517 4726 13598 17457 29702 18119
```

OBSERVATION

The program successfully generated 50 random integer values using `rand()`. The values are displayed in the console.

Table 6-1. Data Generated and Observations

CODE

```
int userInput;

std::cout << "What number to search?";
std::cin >> userInput;

linearSearchArr(max_size, userInput, dataset);
```

OUTPUT

```
What number to search? 5136
Searching is Unsuccessful
Searching Successful!
```

OBSERVATION

Linear search checks each element one by one until the searched value is found. It can be used even when the array is not sorted.

Table 6-2a. Linear Search for Arrays

CODE

```
// Create Linked List for Linear Search
Node<char>* name1 = new_node('J');
Node<char>* name2 = new_node('A');
Node<char>* name3 = new_node('Y');
Node<char>* name4 = new_node('P');
Node<char>* name5 = new_node('E');

name1->next = name2;
name2->next = name3;
name3->next = name4;
name4->next = name5;

// Linear Search Linked List
linearSearchLL('Y', name1);
linearSearchLL('r', name1);
```

OUTPUT

```
Search Successful!
Search Unsuccessfull!
```

OBSERVATION

The linked list is searched sequentially from the first node. M is found in the list, while r is not found.

Table 6-2b. Linear Search for Linked List

CODE

```
int test[8] = {1, 2, 3, 4, 5, 6, 7, 8};

binarySearchAR(8, 12, test);
binarySearchAR(8, 1, test);

return 0;
```

OUTPUT

```
Search element not found
The search element is found!
```

OBSERVATION

Binary search repeatedly divides the sorted array into smaller sections. The value 12 is not found, while 1 is found. The array must be ordered for binary search to work properly.

Table 6-3a. Binary Search for Arrays.

CODE

```
Node<int>* num1 = new_node(1);
Node<int>* num2 = new_node(2);
Node<int>* num3 = new_node(3);
Node<int>* num4 = new_node(4);
Node<int>* num5 = new_node(5);

num1->next = num2;
num2->next = num3;
num3->next = num4;
num4->next = num5;

// Binary Search Linked List
binarySearchLL(num1, 4);
binarySearchLL(num1, 0);
```

OUTPUT

```
Search Successful!
Search Unsuccessfull!
```

OBSERVATION

Binary search for a linked list uses the middle node to reduce the search area. The value 4 is found, while 0 is not found. The linked list must be ordered for the binary search to work correctly.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1:

Array:

Code:

```cpp
#include <iostream>

int main()
{
    int list[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int key = 18;
    int comparisons = 0;

    for (int i = 0; i < 10; i++)
    {
        comparisons++;

        if (list[i] == key)
        {
            std::cout << "Key found at index: " << i << std::endl;
            break;
        }
    }

    std::cout << "Number of comparisons: "
              << comparisons << std::endl;

    return 0;
}
```

Output:

```
Key found at index: 1
Number of comparisons: 2
```

Code:

```cpp
#include <iostream>

struct Node
{
    int data;
    Node* next;
};

int main()
{
    int values[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 1

    Node* head = nullptr;
    Node* tail = nullptr;

    for (int i = 0; i < 10; i++)
    {
        Node* newNode = new Node;
        newNode->data = values[i];
        newNode->next = nullptr;

        if (head == nullptr)
        {
            head = newNode;
            tail = newNode;
        }
        else
        {
            tail->next = newNode;
            tail = newNode;
        }
    }

    int key = 18;
    int comparisons = 0;
    Node* current = head;

    while (current != nullptr)
    {
        comparisons++;

        if (current->data == key)
        {
```

```
        if (current->data == key)
        {
            std::cout << "Key found: "
                      << current->data << std::endl;
            break;
        }

        current = current->next;
    }

    std::cout << "Number of comparisons: "
              << comparisons << std::endl;

    return 0;
}
```

Output:

```
Key found: 18
Number of comparisons: 2
```

Problem 2:

Count of Repeating Instances:

Code:

```cpp
#include <iostream>

int sequentialSearchCount(int list[], int size, int k)
{
    int count = 0;
    for (int i = 0; i < size; i++)
    {
        if (list[i] == k)
        {
            count++;
        }
    }
    return count;
}
int main()
{
    int list[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size = 10;
    int k = 18;
    int count = sequentialSearchCount(list, size, k);
    std::cout << "Search element: " << k << std::endl;
    std::cout << "Number of repeating instances: "
              << count << std::endl;
    return 0;
}
```

Output:

```
Search element: 18
Number of repeating instances: 2
```

Problem 3:

Code:

```cpp
#include <iostream>

int binarySearch(int list[], int size, int key)
{
    int low = 0;
    int high = size - 1;
    int comparisons = 0;

    while (low <= high)
    {
        int mid = (low + high) / 2;
        comparisons++;
        std::cout << "Iteration " << comparisons
                  << ": mid = " << mid
                  << ", value = " << list[mid]
                  << std::endl;
        if (list[mid] == key)
        {
            std::cout << "Key found at index: "
                      << mid << std::endl;

            std::cout << "Total comparisons: "
                      << comparisons << std::endl;
            return mid;
        }
        else if (key < list[mid])
        {
            high = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }
    std::cout << "Key not found." << std::endl;

    return -1;
}

int main()
{
    int list[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int key = 8;
    binarySearch(list, 10, key);
    return 0;
}
```

Output:

```
Iteration 1: mid = 4, value = 11
Iteration 2: mid = 1, value = 5
Iteration 3: mid = 2, value = 6
Iteration 4: mid = 3, value = 8
Key found at index: 3
Total comparisons: 4
```

Problem 4:

Code:

```cpp
#include <iostream>

int recursiveBinarySearch(int list[], int low, int high, int key)
{
    if (low > high)
    {
        return -1;
    }
    int mid = (low + high) / 2;

    std::cout << "Checking index " << mid
              << ", value = " << list[mid]
              << std::endl;

    if (list[mid] == key)
    {
        return mid;
    }
    else if (key < list[mid])
    {
        return recursiveBinarySearch(
            list, low, mid - 1, key
        );
    }
    else
    {
        return recursiveBinarySearch(
            list, mid + 1, high, key
        );
    }
}

int main()
{
    int list[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};

    int key = 8;

    int result = recursiveBinarySearch(
        list, 0, 9, key
    );

    if (result != -1)
    {
        std::cout << "Key found at index: "
                  << result << std::endl;
    }
    else
    {
        std::cout << "Key not found." << std::endl;
    }
    return 0;
}
```

Output:

```
Checking index 4, value = 11
Checking index 1, value = 5
Checking index 2, value = 6
Checking index 3, value = 8
Key found at index: 3
```

Summary of Lessons Learned:

I learned how linear search and binary search work using arrays and linked lists. I also learned that linear search checks the elements one by one, while binary search repeatedly divides a sorted list into smaller parts. I also learned how to modify searching algorithms to count repeated elements and how to implement binary search recursively.

Analysis of the Procedure:

The procedure helped me understand how each searching algorithm works step by step. For linear search, I compared each element with the search key until the key was found. For binary search, I used the middle element and adjusted the searching range depending on whether the key was smaller or larger.

Analysis of the Supplementary Activity:

I was able to determine the number of comparisons needed and observe how binary search can reduce the number of elements checked. The recursive version also helped me understand how a function can call itself to continue the search.

Concluding Statement / Feedback:

I think I did fairly well in this activity because I was able to understand and implement the different searching algorithms. I was also able to verify my answers using the program output. My areas for improvement are writing the code more efficiently, understanding recursion better, and reducing mistakes when implementing the algorithms. The activity helped me improve my understanding of searching techniques

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chat gpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use: **It fixes my grammar but it was my own idea**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.